

is Serpent, which is based on Python. As Python is a very simple and easy-to-understand language, it may be easier to learn in the future. For now though, Solidity is king. A good understanding of it will be very useful when learning other smart contract languages in the future.

What is so special about it?

Solidity is the code behind Ethereum – one of the largest blockchain platforms in the world (the other being Bitcoin). Both do have similarities – like being blockchain platforms with built-in tokens. Bitcoin's currency (Bitcoin) and Ethereum's currency (Ether) are two of the most important and broadly traded cryptocurrencies available. That's where the similarities end. The Bitcoin network was created as a peer-to-peer currency exchange. Bitcoin, as a token, was always intended to be a store of value – an asset. On the other hand, Ethereum was created as a way for people to move anything of value, efficiently, not just cryptocurrency. The token, Ether, was created as a way to pay for those dealings on the platform. Its role was meant to be the 'oil for the network', not an asset like Bitcoin.

Since it was too complex to deal with anything other than cryptocurrencies on the Bitcoin platform, the team at Ethereum created a new system. It required a computing language with far more flexibility than Bitcoin's (largely written in C++); so the team wrote the language Solidity.

What you can do with Solidity

Solidity is the base of the Ethereum network. Without Solidity, there is no Ethereum. Here's what you can do with the former:

1. The programming language gives developers the skill to create their own decentralised apps (dApps). A way to think about it is like the apps you get on your smartphone. You can download all types of applications from the app store, covering everything from games and fitness to banking. dApps are the same. The main dissimilarity is that they are open source. Just as apps in the Apple app store are built to run on iOS, dApps on Ethereum are built to run on Solidity.
2. Within dApps, there are pieces of code recognised as smart contracts. These assist people to swap money, shares, property and nearly anything of value when certain conditions are met. That eliminates the need for an expensive third party like a notary.

So, when can one use Solidity? What are the applications of this programming language?

Solidity is an advanced programming language that has made coding around blockchain platforms particularly easy. Being easy to learn and rather easy to use, Solidity's many applications include the following.

Voting: In the real world, the voting process is vulnerable to many forms of fraud such as the manipulation of data, booth capturing, fake voters, the manipulation of voting

machines, etc. To solve a few of these problems, we could make use of contracts for voting. Solidity can be used to plan the code, and with proper implementation, the process of voting will be smooth, transparent and automatic.

Crowdfunding: If done through contracts, crowdfunding can solve various problems related with commissions for third parties, issues related to managing data, etc. For crowdfunding, smart contracts can work out far better than non-trusted federal systems. These smart contracts can be developed using Solidity.

Blind auctions: Implementation of blind auctions using Solidity is quite simple on Ethereum. An open sale can be created, in which every person is aware of each other's bid. Following this, a blind auction can be designed whereby it will not be possible for anyone to see others' bids until the bidding process ends.

Here is an example of a Solidity program.

```
contract GavCoin
{
    mapping(address=>uint) balances;
    uint constant totalCoins = 1000000000000;

    /// Endows creator of contract with 1m GAV.
    function GavCoin(){
        balances[msg.sender] = totalCoins;
    }

    /// Send $((valueInmGAV / 1000).fixed(0,3)) GAV from the
    account of $(message.caller.address()), to an account
    accessible only by $(to.address()).
    function send(address to, uint256 valueInmGAV) {
        if (balances[msg.sender] >= valueInmGAV) {
            balances[to] += valueInmGAV;
            balances[msg.sender] -= valueInmGAV;
        }
    }

    /// getter function for the balance
    function balance(address who) constant returns (uint256
    balanceInmGAV) {
        balanceInmGAV = balances[who];
    }
}
```

END 

Reference

- [1] <https://en.wikipedia.org/wiki/Solidity>

By Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing. He can be contacted at mehrotra.neetesh@gmail.com.